SPFx packaging, source to .sppkg

A `.sppkg` is a signed zip of bundles and a manifest. Five steps turn source into the file you hand to a tenant admin.

Build chain — the contract

1. Source

`.ts, .tsx, .scss`

- src/webparts/<name>/*.tsx
- webpart.manifest.json
- `tsconfig.json` + eslint
- Node 22 LTS toolchain

2. Bundle

`gulp bundle --ship`

- Webpack split per entry point
- Hashed file names (**.min.js*)
- CSS extracted to its own file
- Source maps stripped for ship

3. Manifests

`gulp bundle --ship (writes)`

- *elements.xml* per web part
- Feature id, version, scope
- Asset paths in SharePoint CDN
- Tokens every property is fillable

4. Package

`gulp package-solution --ship`

- Reads *package-solution.json*
- Zips *sharepoint/* folder
- Generates AppManifest.xml
- Signs with cert & timestamp

5. .sppkg

`*.sppkg artefact`

- upload to App Catalog
- deploy / retry / revoke
- ID matches package GUID
- version label is trustworthy

What is inside a `.sppkg`?

```
$ unzip -l hronboard.sppkg
AppManifest.xml  package descriptor + permissions
  sharepoint/elements.xml  feature / web part entries
  sharepoint/assets/*.js  bundled, hashed, scoped
  sharepoint/assets/*.css  extracted stylesheet
[Content_Types].xml  Office add-in friendly MIME map
manifest.zip.sig  CAB-style signature block
```

- A *.sppkg* is just a signed zip — rename to *.zip* and unzip it to inspect
- Bundle paths embed the CDN host at build time — changing CDN breaks existing web parts
- The AppManifest.xml is the large package metadata tenant admin sees
- SPFx 1.21+ baseline: Node 22 LTS, React 18, TypeScript 5.4+

package-solution.json, line by line

The single JSON the build trusts. Read every key before you bump the version.

config/package-solution.json

```
// 4-segment version, GUID matches across every environment
```

```json
{
  "$schema": "https://developer.microsoft.com/json-schemas/spfx-build/package-solution.schema.json",
  "id": "d2b9f33e-7e62-4c0e-9f43-1b15b87c4f72",
  "version": "1.2.0.0",
  "includeClientSideAssets": true,
  "isDomainIsolated": false,
  "developer": {
    "name": "Contoso HR Engineering",
    "websiteUrl": "https://contoso.com",
    "privacyUrl": "https://contoso.com/privacy",
    "termsOfUseUrl": "https://contoso.com/terms",
    "mpnId": "1234567"
  },
  "features": [{
    "title": "hr-onboarding",
    "id": "e9d6cf5a-9c40-4f4b-9a4d-1b1ea4df63dd",
    "version": "1.2.0.0",
    "assets": {
      "elementManifests": ["elements.xml"],
      "clientSideComponentAssets": [
        "https://cdn.contoso.com/hr-onboarding/*"
      ]
    }
  }],
  "webApiPermissionRequests": [{
    "resource": "Microsoft Graph",
    "scope": "Sites.Selected"
  }]
}
```

What each block does

id · stable GUID
Same across dev/test/prod. Re-using it auto-upgrades; changing it is a brand-new install.

version · 4-semver
SharePoint reads the first three only — keep “0” for the build index.

developer · trust signal
Surfaced on the App Catalog tile. *mpnId* only matters if you ship to other tenants through AppSource.

features · element list
One feature per web part or extension. *feature id* differs from the *package id* — SharePoint tracks them separately.

webApiPermissionRequests · consent
Admin sees these on installation in the SharePoint admin center — keep it to the minimum you actually need.

clientSideComponentAssets · CDN
Wildcard URL is what SharePoint fetches at page render. Wrong host = white-page web part.

App Catalog: tenant vs site collection

Where your `.sppkg` lives. One tenant-wide library or one per site collection — never both for the same id.

Tenant App Catalog

/sites/apps, the canonical home

- One per tenant, lives at */sites/apps*
- Apps here can be deployed tenant-wide
- Owner: SharePoint admin (or delegated)
- First choice for any organisation-wide product
- Holds up to **50 000 files, 250 MB each**
- Syncs the bundle to tenant CDN automatically
- Required destination for SPFx packages

```
// create from SharePoint admin center
m365 spo sites appcatalog ensure
m365 spo app add --filePath ./sharepoint/solution/hronboard.sppkg
m365 spo app deploy --id d2b9f33e-7e62-4c0e-9f43-1b15b87c4f72
m365 spo app install --id d2b9f33e-7e62-4c0e-9f43-1b15b87c4f72
```

Site Collection Catalog

/AppCatalog, scoped per site

- Lives at */sites/<x>/AppCatalog* per collection
- Updated independently for that site only
- Useful for LOB apps, locked-down tenants
- Owner: site collection admin, not tenant admin

Decision rule of thumb

- **Tenant** when every site — or most sites — need the same web part
- **Site collection** when only the HR subsite uses it and tenant admin is off-limits
- **Never both** for the same package id — the second upload fails
- **Migration path**: deploy tenant-wide later for broader use
- **Storage**: tenant catalog is the only one Microsoft support relies on

Catalog settings and the CDN toggle

Three flags in the admin center decide whether your app is review-free, auto-deployed, or synced via CDN.

Apps that don't require review

Skip the Microsoft 365 app compliance scan for new uploads. **First-party in-house apps that never leave the tenant** should live behind this. Turn it off for AppSource candidates.

Tenant-wide deployment

Enable on the package detail page in the App Catalog. App installs to every site collection. Sites added later get it on first request.

Public CDN host

`publiccdn.sharepoint.com`

- Anonymous read by design
- Default for SPFx bundles
- Fastest — no token round trip
- Lock down with Conditional Access? No.
- Fits bundles that hold no secrets

Private CDN host

`privatecdn.sharepoint.com`

- Auth required — carry bearer token
- Pick when bundle touches confidential data
- Conditional Access policies *do* apply
- Slightly slower per request
- Mostly used for tenant-private assets

Upload → deploy → consent → sync



CDN sync · bundle goes live

`cdn.sharepoint.com / <tenant-prefix> / hr-onboarding / *`

Usually < 5 min; force a re-publish if a stale bundle lingers

```
// PnP PowerShell: enable CDN, set Origin URL
Set-PnPTenantCdnEnabled -CdnType Public -Enable $true
Add-PnPTenantCdnOrigin -CdnType Public -OriginUrl /sites/apps/ClientSideAssets
Publish-PnPAppPackage -Path ./hronboard.sppkg -Force
Install-PnPApp -Identity d2b9f33e-7e62-4c0e-9f43-1b15b87c4f72 -Force
```

Deployment targets and CDN strategy

Once the package lands in the catalog, where do its bytes live — and what breaks when you move them?

From `.sppkg` to a working web part — every hop

1. Package

- Built by gulp
- Hashed bundles inside
- Signed locally
- ~ 4 MB typical

2. App Catalog

- /sites/apps/CDN
- Locked from the tenant side
- Permissions requests listed
- Tenant-wide toggle lives here

3. Tenant CDN

- publiccdn.sharepoint.com
- Fast, anonymous
- 5-min propagation
- No external domain needed

4. Page render

- SPFx loader fetches bundle
- Lazy, manifest-keyed
- Cached for one hour
- Resilient to CDN outage

5. Web part

- React renders
- PnPjs + Graph fire
- User sees data
- Logs to App Insights

Three CDN options — which one fits this web part?

Public tenant CDN

`publiccdn.sharepoint.com`

+ Default for SPFx, zero config
+ Global edge, free egress
+ No auth round trip — fast
- Anyone can fetch the bundle
- Conditional Access does not apply

Private tenant CDN

`privatecdn.sharepoint.com`

+ Bearer-token protected
+ Conditional Access honoured
+ Pick when payload is sensitive
- Token round trip — slower
- Bundle paths still hard-coded

Azure Blob / Front Door

`cdn.contoso.com`

+ Full control, WAF + policies
+ Brand-aligned host
+ Use for non-SharePoint assets
- Push manifests need rebuild
- Not the SPFx default — opt-in

CLI & PnP PowerShell — ship the package

Two CLIs do the same job. Pick one per environment — never “browser and CLI” for the same package id in the same day.

CLI for Microsoft 365

```
$ npm i -g @pnp/cli-microsoft365
```

```
// 1. sign in once per shell
m365 login --authType deviceCode

// 2. ensure the tenant App Catalog exists
m365 spo sites appcatalog ensure

// 3. upload the package
m365 spo app add \
  --filePath ./sharepoint/solution/hronboard.sppkg \
  --overwrite

// 4. deploy + install tenant-wide
m365 spo app deploy \
  --id d2b9f33e-7e62-4c0e-9f43-1b15b87c4f72
m365 spo app install --id d2b9f33e-... --scope tenant

// 5. confirm the bundle reached CDN
m365 spo app get --id d2b9f33e-... --withCdnInfo
```

PnP PowerShell

```
Install-Module -Name PnP.PowerShell
```

```
# connect, register your Entra app
Connect-PnPOnline -Url https://contoso-admin.sharepoint.com \
  -Interactive -ClientId <app-id> -Tenant contoso.onmicrosoft.com

# publish & install in two steps
Add-PnPApp -Path ./sharepoint/solution/hronboard.sppkg -Publish
Install-PnPApp -Identity d2b9f33e-... -Scope Tenant -Force

# deploy tenant-wide (banner flag)
Update-PnPApp -Identity d2b9f33e-... -Deploy

# revoke, then uninstall
Update-PnPApp -Identity d2b9f33e-... -Uninstall
Remove-PnPApp -Identity d2b9f33e-... -Force

# confirm CDN sync (returns CDN host)
Get-PnPApp -Identity d2b9f33e-... -IncludeCdnInfo
```

Versioning and upgrade strategy

Two version stamps ride along — SharePoint reads the first three, the build index lets you hotfix without rejecting.

Two stamps — keep them in lockstep

package.json · source-of-truth

```
"version": "1.2.0"
```

- Semver: **major.minor.patch** + build index
- Pipeline reads it; lock it before deploy
- Tag the git commit with the same version

package-solution.json · the stamp SharePoint reads

```
"version": "1.2.0.0"   "features"[0].version: "1.2.0.0"
```

- 4 segments; first three must match package.json
- Feature id stays the same across upgrades
- Lower or equal version = rejected — no rollback

What triggers an automatic upgrade

`tenant-wide deploy on = bump first three segments`

- Major — breaking schema, rename, drop property
- Minor — new feature or webhook surface added
- Patch + build index — bundle hash change, no schema delta
- Build index alone is hotfix — legal but rarely triggered
- Any segment decrement = rejected on upload

Upgrade flow in four steps

1. Code change

Branch, PR, merge. Tests gate this — if red, the version is not bumped.

2. Bump

```
npm version 1.2.1
```

3. Repackage

```
gulp bundle --ship && gulp package-solution --ship
```

4. Upgrade

```
Install-PnPApp ... -Scope Tenant
```

Data model — no migration hooks in SPFx

- SPFx itself has *no* upgrade migrations for lists or libraries.
- Cheap lifts — column renames or defaults — run from a one-shot PnP PowerShell job.
- Larger lifts — use a PnP provisioning *template.xml* tied to the version tag.
- Always gate the schema job behind a fence — new version installed means job fired.

Two versions in the same tenant

Same package id auto-upgrades. To run v1 and v2 in parallel you need a different package id — and a clear migration plan.

Same id, auto-upgrade

`default tenant-wide behaviour`

- New package is same id, higher version
- Every instance upgrades in place
- Last writer wins — quick
- Property bag values are preserved
- No rollback path on the same id
- Old page-level config can vanish

New id, parallel universes

`align two web parts in same page`

- Ship v2 as a brand-new package id
- Add alongside v1 to compare in pilot
- Use a feature flag in your web part code
- Push v1 to read-only mode; cut over
- Two bundles in CDN — watch payload
- Doubles your security review surface

Staged tenants

`dev / uat / prod tenants`

- Run v_next in dev only
- Promote when UAT signs off
- Single prod tenant = single source
- Sample data lives in dev
- Tenant overhead, licences
- Trust the staging mirror, not test

Schema migration — one-shot, fenced to the version

```
// PnP PowerShell: rename a column on every list of a template
$site = Connect-PnPOnline -Url https://contoso.sharepoint.com/hr -Interactive
$lists = Get-PnPList | Where TemplateId -eq 100
foreach ($l in $lists) {
  $f = Get-PnPField -List $l -Identity "OldName" -ErrorAction SilentlyContinue
  if ($f) { $f.Title = "NewName"; $f.Update() }
}

// tag in the version named-values column
Set-PnPPropertyBagValue -Key "hronboard_schema_version" -Value "1.2.0"
```

- Runs **once** per up version — idempotent on retry
- Record the version in a property bag entry so a re-deploy no-ops
- For big lifts, ship a PnP provisioning *template.xml* with the package
- Run inside a *site*, not *tenants*, unless you really need it everywhere
- If a migration fails, block the upgrade — never install on partial schema

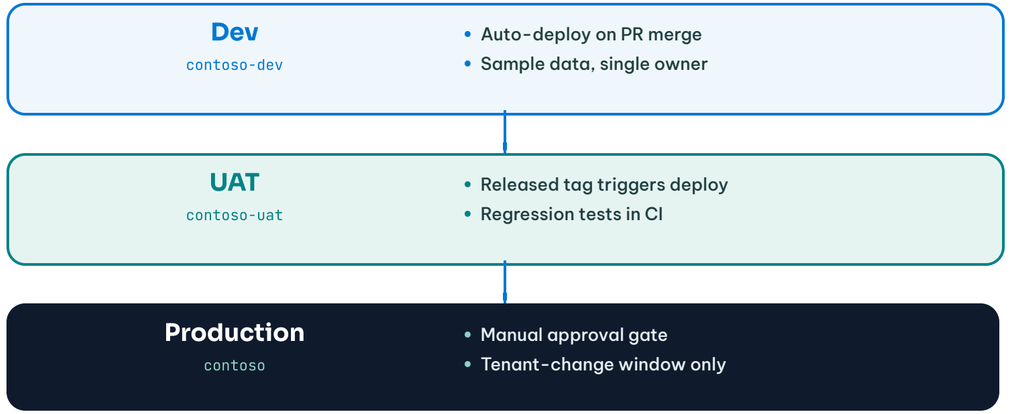
ALM environments & source layout

One repo per solution. Three tenants for the lifecycle. Branch protection that ties the env to the version stamp.

/hronboard repo · one solution, many layers

```
// monorepo, yarn workspaces
hronboard/
  .github/workflows/  ci, deploy, smoke
  azure-pipelines/  azure-pipelines.yml
  src/spfx/  web part + 2 extensions
  src/functions/  Azure Function Graph triggers
  src/flows/  Power Automate solution.json
  src/infra/  Bicep templates + scopes
  src/pnp/  Pnp.PowerShell deployment scripts
  src/provisioning/  PnP template.xml per list
  tests/spfx/  jest + @spfx/assert patterns
  scripts/  version-bump, pr-check
  package.json  workspaces root, Node 22 LTS
  engines.json  pnpm 9, Node 22.x
  VERSION  single source: 1.2.0
```

Three tenants, three roles



Branch protection & PR gates

Branch

- *main* protected, no direct push
- feature/* via PR — squash
- release/* cuts tags
- hotfix/* -> main + tag

Code owners

- *CODEOWNERS* lists each layer
- SPFx: SPFx lead approves
- infra: cloud-arch approves
- Identity: security team

Required checks

- *jest* — SPFx unit tests
- *eslint + prettier*
- secret scan (gitleaks)
- pipeline dry-run deploy

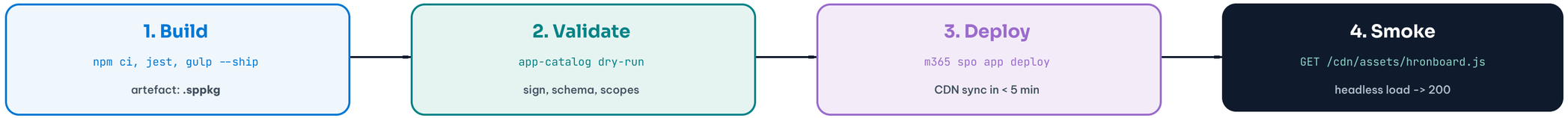
Approvals

- Release — 2 of 4 approvers
- Hotfix — 2 of 4
- Schema migration — DBA
- Security review for scopes

Pipeline stages — build, validate, deploy, smoke

Four stages, two CI vendors, identical semantics. Pin Node 22, cache gulp artefacts, do not gate prod on Friday afternoon.

Four sequential stages — hand off a real artefact between them



GitHub Actions · deploy job

```yaml
# .github/workflows/deploy.yml
jobs:
  build:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run build -- --ship
      - uses: actions/upload-artifact@v4
        with: { path: sharepoint/solution/*.sppkg }

  deploy:
    needs: build
    environment: ${{ matrix.env }}
    steps:
      - uses: actions/download-artifact@v4
      - uses: pnp/action-cli-microsoft365@v1
      - run: m365 spo app deploy --id ${{ secrets.PKG_ID }}
```

Azure DevOps · equivalent stages

```yaml
# azure-pipelines/deploy.yml
trigger:
  branches: { include: [main, release/*] }
stages:
  - stage: Build
    jobs:
      - job: compile
        pool: { vmImage: ubuntu-22.04 }
        steps:
          - task: NodeTool@0
            inputs: { versionSpec: 22.x }
          - script: npm ci && npm run build -- --ship
          - publish: sharepoint/solution/*.sppkg
  - stage: Deploy
    dependsOn: Build
    jobs:
      - job: ship
        - task: PnP.PowerShell@1
          script: Install-PnPApp ... -Scope Tenant
```

Least privilege scopes and Sites.Selected

Pick the smallest scope that still works. *Sites.Selected* replaces the temptation to ask for *Sites.ReadWrite.All*.

Ask in the smallest scope — `package-solution.json`

```
// scopes that line up with the work the web part does
"webApiPermissionRequests": [
  {
    "resource": "Microsoft Graph",
    "scope": "Sites.Selected"
  },
  {
    "resource": "Microsoft Graph",
    "scope": "User.ReadBasic.All"
  },
  {
    "resource": "Microsoft Graph",
    "scope": "Mail.Send"
  }
]
```

Scope ladder — start at the bottom

Sites.Selected	per-site grant
User.ReadBasic.All	every user, no PII
Mail.Send	on behalf of user
Sites.ReadWrite.All	tenant-wide writes
avoid <role>.All	security review needed

Sites.Selected — grant per site

- App acts only on sites the admin lists
- Grant in PowerShell:
- Read · Write · Owner roles
- Revoke — *DELETE* same endpoint
- Document every granted site id

Conditional Access alignment

- MFA — enforce for admins at least
- Device compliance for field workers
- Block legacy auth at the tenant
- Named locations for known office IPs
- Risk-based sign-in prompts on

Admin consent review

- Review every *webApiPermissionRequests*
- Reject overly broad scopes
- Match scope to feature — not future
- Document revocation procedure
- Service tree owner signs off

Hardening: Key Vault, CSP, CORS, tokens

The four hidden cracks the breach report always names: leaked config, broad CSP, cross-origin write, and accepted every token.

CSP · config/write-manifests.json

```
// strict CSP — no eval, no wildcards on connect
{
  "script-src": "'self' 'unsafe-inline' *.sharepoint.com",
  "connect-src": "'self' graph.microsoft.com *.contoso.com",
  "img-src": "'self' data: blob:",
  "frame-src": "*.sharepoint.com *.office.com",
  "object-src": "'none'",
  "default-src": "'none'",
  "base-uri": "'self'",
  "form-action": "'self'",
}
```

Key Vault · Azure Function

```
// bound at start-up; never logged, never echoed
import { SecretClient } from "@azure/keyvault-secrets";
import { DefaultAzureCredential } from "@azure/identity";

const kv = new SecretClient(
  process.env.KEYVAULT_URI!,
  new DefaultAzureCredential());
const { value } = await kv.getSecret("hronboard-smtp");
return { value };

// managed identity on the function app — no secret in code
```

Secrets

Key Vault + managed identity

- Never in repo, ever
- *gitleaks* in CI gate
- Rotate every 90 days
- Log the access, not the value

Content security

strict, no wildcards

- *default-src 'none'*
- Allow per host, not per *
- Hashed inline scripts when possible
- Revisit after every dependency add

CORS

cross-origin write, narrow

- *Access-Control-Allow-Origin*
- Specific host, not * (with creds)
- Preflight cache to 5 min
- Function exposes only the verb you need

Token handling

acquire, scope, refresh, revoke

- Always use *MSGraphClientV3* login
- Acquire silently when refreshed
- Honour 401 — never swallow
- Cache by fingerprint, not the JWT

Debug and diagnose in production

Repro the live page with two query params. Match the symptom before you change one byte of code.

Debug surfaces — where to look first

A. Workbench

```
gulp serve --nobrowser  ⇒  https://contoso.sharepoint.com/_layouts/workbench.aspx
```

Mocked *pageContext*, mock *MSGraphClientV3*. Pure logic — no data plane calls.

B. Hosted workbench

```
?debug=true & ?loadSPFX=true
```

Same tenant, real tokens, real lists — half the friction of a full deploy.

C. Live page

```
?loadSPFX=true · bundle re-mounted from /tempworkbench
```

Test on a real page, never edit *production* through this URL — only *dev*.

D. DevTools + App Insights

```
F12 console, Application Insights Live Metrics, Service Health
```

TraceId flows through to logs. Service Health tells you whether it is you.

Read the symptom — common signals

Symptom	Likely cause
`white-page web part` bundle 404 at CDN	CDN host switched after build; redeploy with correct host
`token 401` silent redirect?	Admin revoked consent; surface a clear re-consent prompt
`package "stuck deploying"` App Catalog state	Lower-version upload; cancel job and bump the package semver
`CORS preflight fails` function vs page	Function origin not listed in Function App CORS or CSP connect-src
`Graph 429` sharepoint tenant	Throttling — retry with backoff, cache locally, off-peak
`manifest mismatch` id vs id collision	Two packages share an id; rename the older one and reinstall

Rule of thumb

Confirm *Service Health* first — if Microsoft is fine, the bug is in your last deploy. Re-deploy last known good, then bisect.

Three failures that never send a notification

No alert fires. The page renders blank for one user, the sync silently stops for everyone else.

01 · Subscription expires

180-day ceiling, no e-mail

Graph */subscriptions* times out after 6 months. The handler goes quiet — no traffic, no error, no event.

```
// renew 7 days early, every check
await graph.api(
  `/subscriptions/${id}`
).update({
  expirationDateTime: next
});
```

Catch — Timer trigger renews

Daily Job, save the id list, never trust the live set

02 · Stale bundle still cached

CDN swaps, browsers keep warm cache

Hotfix deploys but a 7-day active session keeps the old bundle in *Cache-Control*. Page renders as if nothing changed.

```
// public CDN, set short max-age on hotfix
Set-PnPTenantCdnPolicy
  -CdnType Public
  -IncludeFileExtensions "js,css"
  -MaxAge 60  # 1 min TTL
  -Policy "hotfix"
```

Catch — version-stamp every URL

Hashed names are not optional; cache-control flips per package

03 · Wrong id slips in

same id, wrong package body

Two teams share *package id*. Tenant-side accepts both. Property configs silently overlay — users see a half-baked web part.

```
// CI asserts id matches the approved regex
$pkgId = (jq -r .id pkg.json)
$match = $pkgId -match '^hr-onboarding-[a-f0-9]{8}-'
$pkgId = (jq -r .id
  sharepoint/solution/*.sppkg)
assert $pkgId -eq $expected
```

Catch — central id registry

App Catalog GUID inventory, PR-reviewed list of who owns what

The rule — if a thing can go silent, instrument it now

cron renew + hash bust + id registry + a dashboard that screams when the count drops

Silent failures kill harder than noisy ones because nobody pages. Wire a heartbeat for every state that matters.

To detect, define no flow

If you cannot describe what *success* looks like, you will not notice failure.

Production go-live, on the checklist

Ship small. Ship observed. Ship when the runbook is tested, the rollback works, and the dashboard reads green.

Seventeen gates — tick them, then ship

#	Item	Owner
01	UAT sign-off, documented in PR	UAT lead
02	Version stamp + tenant id pin in code	Release eng
03	Smoke test passed on packaged bundle	CI pipeline
04	App Insights live + custom events wired	Observability
05	CSP / CORS / scopes reviewed again	Security review
06	Roll-back script tested in UAT	Release eng
07	Service Health dashboard green	SP admin

Window & speed rules

- Tuesday or Wednesday only — never Friday
- 10:00-14:00 local — not in last 90 min of support hours
- Pilot ring (5% tenants) for ≥ 24 h before full ring
- Cap the change at ≤ 4 h, pre-stage traffic spike expectations
- Freeze right after — no other tenant changes for 48 h

Rollout rings

Ring 1 · Pilot

`5% sites · 24 h soak`

Internal champions file feedback. No POs allowed.

Ring 2 · Early

`20% sites · 48 h soak`

Volunteer business units. Performance traces reviewed.

Ring 3 · Tenant

`100% · feature flag`

Flip on from admin center. Monitor set alerts.

Incident response — first 30 minutes

- Acknowledge — page the on-call
- Reverse last ring (instant)
- Open Service Health
- Grab correlation id from logs

Monitoring and config drift

Shipped is not done. Two things still bite — telemetry the dashboard reads, and tenant config drifting from the desired state.

Application Insights · from the web part

```
// tip: use the @microsoft/sp-application-insights package
import { ApplicationInsights } from
  "@microsoft/applicationinsights-web";

const ai = new ApplicationInsights({
  config: {
    instrumentationKey: ctx.pageContext.legacyPageContext
      .azureAppInsightsInstrumentationKey,
    enableAutoRouteTracking: true
  }
});
ai.loadAppInsights(); ai.trackEvent({ name: "hronboard_launch" });
```

Microsoft 365 DSC · restore the desired state

PowerShell Desired State Configuration for the tenant

```
# drift detected on SPOTenantCdnEnabled
Start-DscConfiguration
  -Path ./m365dsc/contoso
  -Wait

# daily CI diff against Git-tracked MOF
Test-DscConfiguration  -Detailed |
  Export-Clixml ./artifacts/drift.xml

# recover: re-run the desired config in prod
Invoke-DscResource -Method Set ...
```

Page metrics

browser-side telemetry

- Render time p50/p95
- Bundle fetch time
- JS errors per session
- Uncaught rejections

Graph calls

success, latency, 429

- Endpoint + status code
- Throttling ratio per hour
- Auth refresh failures
- Top N sites by traffic

Tenant drift

CDN, settings, scopes

- Public CDN still enabled?
- App catalog ownership
- Conditional Access
- Revoked scopes, no orphans

User impact

adoption, sentiment

- Daily active web parts
- Help-desk tickets
- Recall/remove ratio
- Survey NPS quarterly

Capstone — HR Onboarding Tracker

Ship one product touching every surface we covered this week. Real users, real data, real ALM.

Every layer comes from this week's five days

Day 1 · Onboard list

- Site, list, content type
- Permission inheritance
- Graph read fallback

Day 2 · SPFx web part

- React, property pane
- Per-user config
- Field Customizer for status

Day 3 · Data + Graph

- PnPjs manager view
- Graph /users for photos
- Cache delta query

Day 4 · Extensions + Flow

- Header banner
- Power Automate approval
- Function to re-index

Day 5 · Ship it

- Package + deploy
- Pipeline gates
- Monitor + drift

What HR asks the solution to do

- Track every new hire by week, manager, and role
- Surface a banner with current week & outstanding checklist items
- Trigger a Power Automate approval before status changes
- Re-index search whenever a colleague is added
- Restrict to a Sites.Selected grant so HR can't read engineering
- Telemetry on adoption · graceful degradation when offline

What you deliver

1. `hronboard.sppkg`

   installed tenant-wide, version 1.2.0, Sites.Selected + Mail.Send

2. `hronboard-flow.json`

   approval flow exported, deployed via Power Automate solution

3. `hronboard-fn.zip`

   Azure Function published, App Insights instrumentation key set

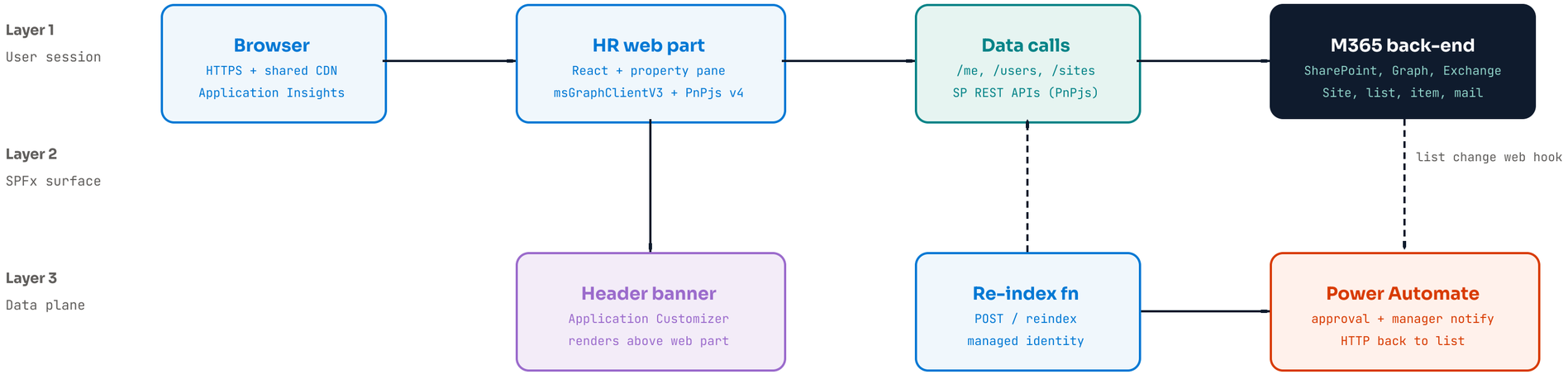
4. **Pipeline + DSC state**

   GitHub Actions OR Azure DevOps YAML + M365 DSC MOF checked in

Wiring — every box from every day

Browser talks to the web part. Web part talks to data and to a function. Flow reacts to changes and keeps search honest.

Request flow — user lands on the Onboarding page



Identity, scope, secrets — the supporting rails

Entra ID
- SPA delegates
- Sites.Selected + Mail.Send
- Admin consent once

Key Vault
- SMTP creds for Flow
- App Insights key
- managed identity reads

App Catalog
- tenant-wide deploy
- CDN sync < 5 min
- *hronboard.sppkg*

Pipeline
- build + sign + sync
- deploy on tag
- smoke → App Insights

Lab — ship the prototype end-to-end

Six ordered steps. Copy the commands, run them, fix the first error message. Resist the temptation to skip ahead.

01 Scaffold + provision

```
yo @microsoft/sharepoint
m365 spo sites appcatalog ensure
m365 spo list set --web hr --listTitle Onboard
  --DefaultSensitivityLabel Confidential
m365 spo user ensure -userName hronboard
```

Out:

- App Catalog ready at */sites/apps*
- HR site + Onboard list + hronboard user

02 Add scopes + consent

```
// patch package-solution.json
"webApiPermissionRequests": [
  { resource: "Microsoft Graph",
    scope: "Sites.Selected" },
  { ..., Mail.Send } ]
```

Out:

- Admin sees scopes in admin center
- *Sites.Selected* + *Mail.Send* requested

03 Build + publish

```
npm ci
npm run build:fast # one-time local sanity
gulp bundle --ship
gulp package-solution --ship
git tag v1.2.0 && git push --tags
```

Out:

- *hronboard.sppkg* signed
- Tag triggers the pipeline

04 Deploy tenant-wide

```
m365 spo app add \
  --filePath ./sharepoint/solution/hronboard.sppkg
m365 spo app deploy --id $sppkgId
m365 spo app install --id $sppkgId --scope tenant
rm web part from a test site, drag in
```

Out:

- Banner visible on every modern page
- Web part appears in the toolbox

05 Wire flow + function

```
m365 pa solution import \
  --path ./src/flows/hronboard.zip
func azure functionapp publish hronboard-fn
Set-PnPAppPermission -AppId $sppkgId -Site $hrUrl
  -Permission "write"
```

Out:

- Approval flow starts on item change
- Function reachable, identity-bound

06 Smoke + monitor

```
curl -fSL https://$cdn/hronboard/hronboard.js
curl -fSL POST -d '{"listId":"..."}'
  $funcUrl/reindex
m365 spo app get --id $sppkgId --withCdnInfo
ai.LiveMetrics · Dsc drift report
```

Out:

- Both 200; CDN + function healthy
- Lifecycle checks logged

Acceptance criteria and grading rubric

You ship when every must-do line passes. Stretch items shift the grade, not the go-live decision.

MUST Pass to ship

- `hronboard.sppkg` builds and deploys without errors
- App installs tenant-wide on the contoso-uat tenant
- *Sites.Selected* + *Mail.Send* granted by admin
- Banner appears on every modern page
- Approval flow starts on item status change
- Function re-indexes within 60 seconds
- App Insights live metrics show the launch event

SHOULD Earn the grade

- Pipeline runs from a clean checkout
- Field Customizer shows status pill inline
- CSP forbids *'unsafe-eval'* in this app
- Secrets retrieved from Key Vault, no literals
- Feature flag gates the new column for ≥ 24 h
- DSC daily drift report committed to repo
- Rename + install demo works in UAT

STRETCH Earn a star

- Two-version split — parallel universes
- Multi-tenant App Catalog distribution
- Conditional Access policy for the app
- Automated dark-skip smoke in CI
- Footer in private CDN for tenant brand
- Real user study with two HR managers

Delivery timeline — two weeks, three rings

Week 1 · build

Mon — scaffold, contract

Wed — data + banner

Fri — UAT deploy, smoke

Deliverable: *v1.0 in UAT*

Week 2 · rings

Mon — pilot ring, 5% of sites

Wed — 20% early adopters

Fri — hold for stability report

Deliverable: *v1.1 soak clean*

Week 3 · go live

Tue 10:00 tenant-wide

Wed — freeze, monitor

Thu — retro + handoff

Deliverable: *v1.2.0 in prod*

Review on the day

- Demo on a fresh tenant
- Run a tracer bullet deploy
- Read the App Insights
- Discuss a roadmap item